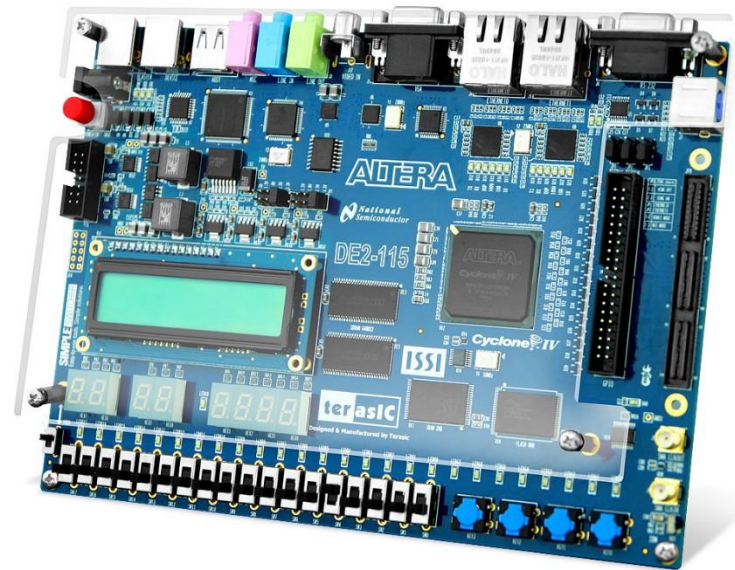


COEN-230

Digital System Design I

Dr. Fadi Alzhouri

Department of Electrical and Computer Engineering
Gust University





Outline

Topics (Ch. 5):

Logic circuits that can store information

- Flip-Flops (single bit)
- Registers (multiple bits)
- Counters



A new class of circuits

So far:

Combinational circuits where the value of each output depends only on the values of signals applied to the inputs.

New class of circuits

The output of the circuit depend not only on the present values of the inputs but also on the **past** input of the circuit.

These circuits include storage elements that represent the **state** of the circuit.

These circuits move from state to state as input changes.

Called **Sequential**.

Consists of combinational circuit and feedback circuit

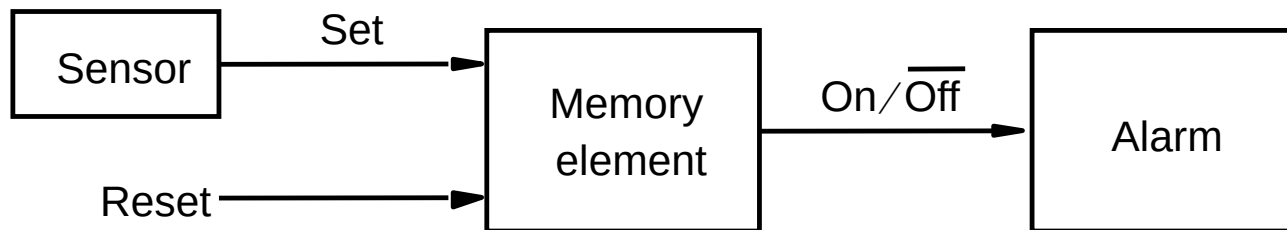


Why do we need memory?



Alarm clock example:

- On when the on/off signal = 1
- Off when the on/off signal = 0



Control of an alarm system.

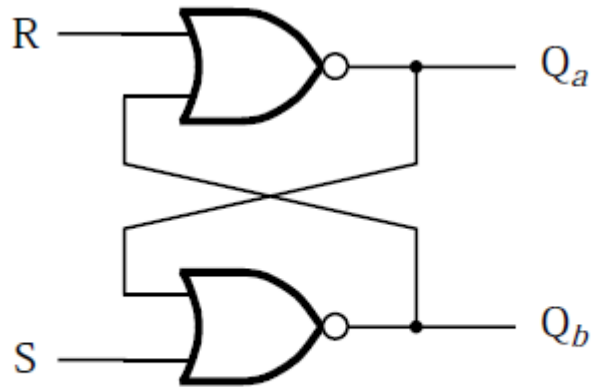


Set & Reset are user-controlled input signals



1- Basic Latch: SR Latch

- Cross-coupled design style
 - Initially, Q_a & Q_b are complements of each other



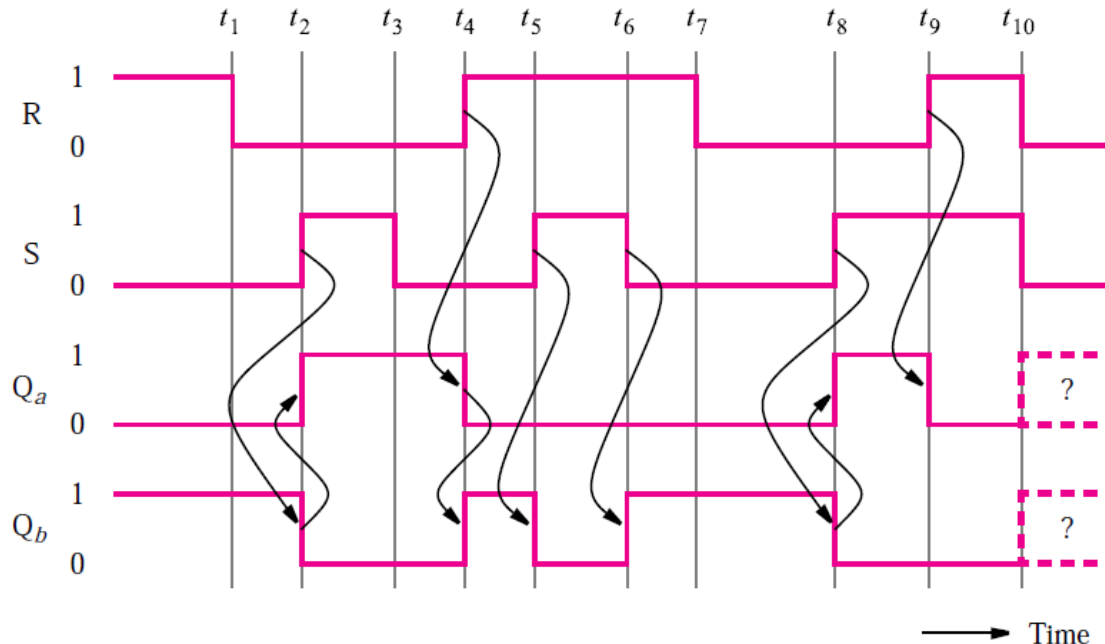
(a) Circuit

S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change)
0	1	0	1	
1	0	1	0	
1	1	0	0	(Not allowed)

(b) Characteristic table



Basic Latch (SR Latch)



- The behavior is analyzed using a “**Timing Diagram**”. If R & S change from 11 to 00 at the same time, oscillation occurs
 - Q_a & Q_b go to 11 then to 00 then 11 ...etc



Gated SR Latch

SR latch can serve as a memory element since it remembers its state when both the S and R inputs are 0.

It changes its state in response to changes in the signals on these inputs.

The state changes occur at the time when the changes in the signals occur.

If we cannot control the time of such changes, then we don't know when the latch may change its state.

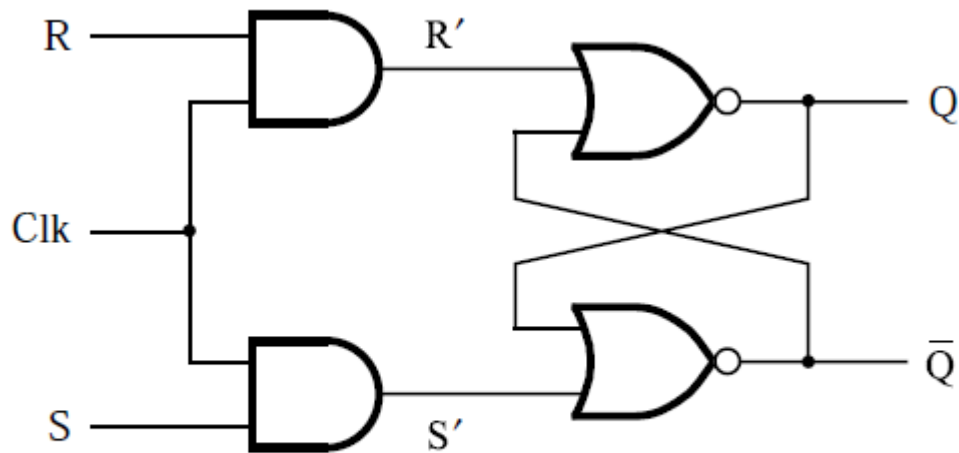
Need an ENABLE signal so that latch responds to input signals only when $\text{ENABLE} = 1$

How can we modify the basic latch?



2- Gated SR Latch

First use of Clock (Clk) signal! (similar to Enable) :
Enables saving to latch when **clock is 1**



(a) Circuit

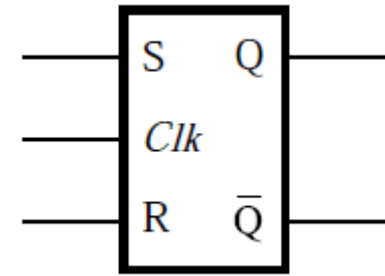
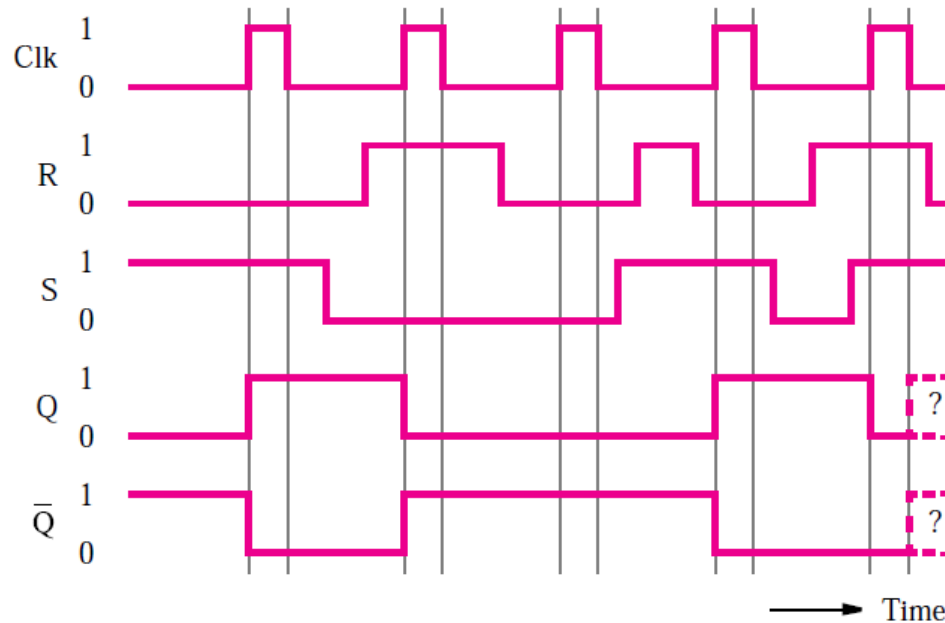
Clk	S	R	$Q(t+1)$
0	x	x	$Q(t)$ (no change)
1	0	0	$Q(t)$ (no change)
1	0	1	0
1	1	0	1
1	1	1	x (Not allowed)

(b) Characteristic table

- Oscillation problem still exists
 - S & R should not go from 11 to 00 while Clk is equal to 1



Gated SR Latch – Timing diagram



Notice that Clk signal occurs in regular intervals, as is the case of clock signal in real life systems

Use of a clock simplifies logic design significantly



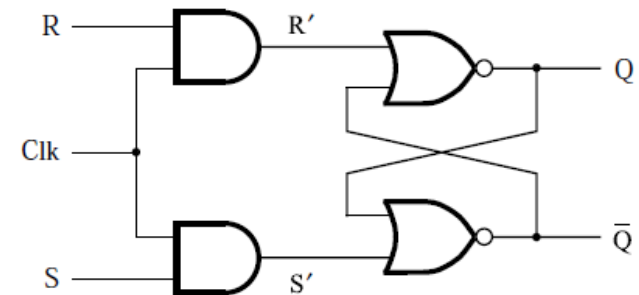
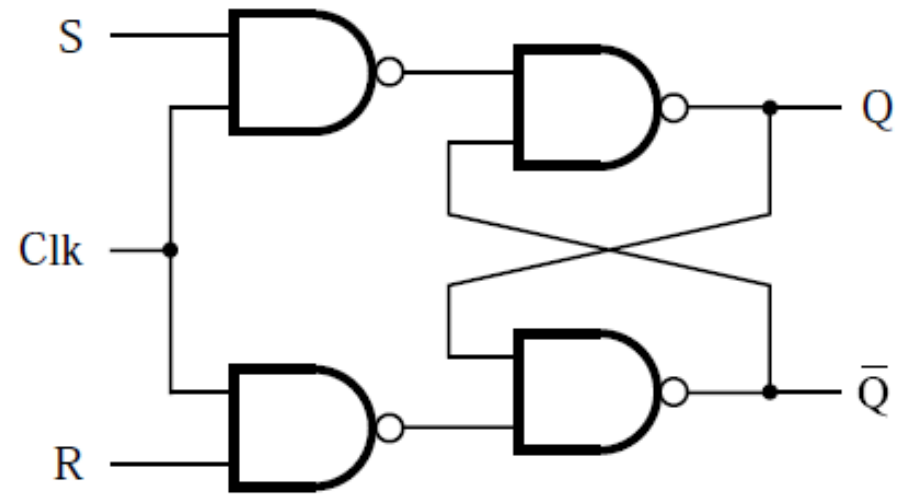
3- Gated SR Latch (using NANDs)

SR latch can be implemented using NAND gates

Note how S & R signal are connected to different gates

Why do we like this implementation over ANDs/NORs?

Simpler design (cost effective)

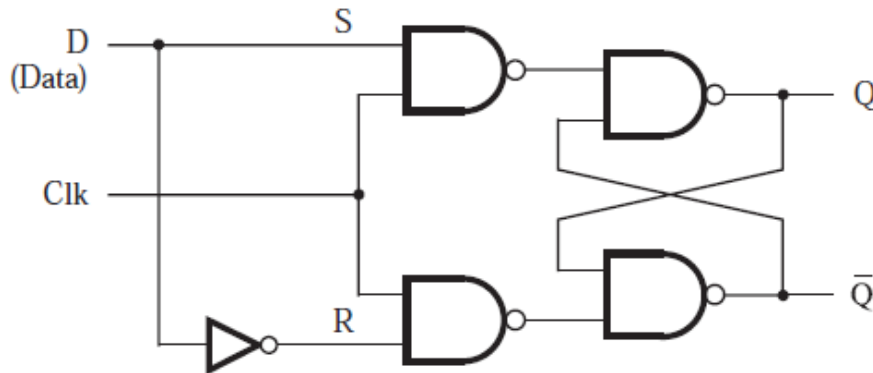




4- Gated D Latch

Built using gated SR latch

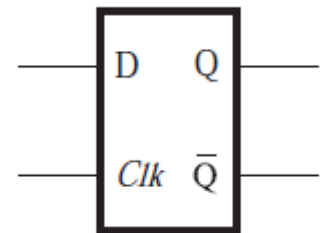
- It simplifies use of latches as storage elements



Clk	D	$Q(t + 1)$
0	x	$Q(t)$
1	0	0
1	1	1

● No oscillation problem! Why?

- Very useful memory element in practice
- The latch is “**level-sensitive**” because the output is controlled by the level of the Clk input.

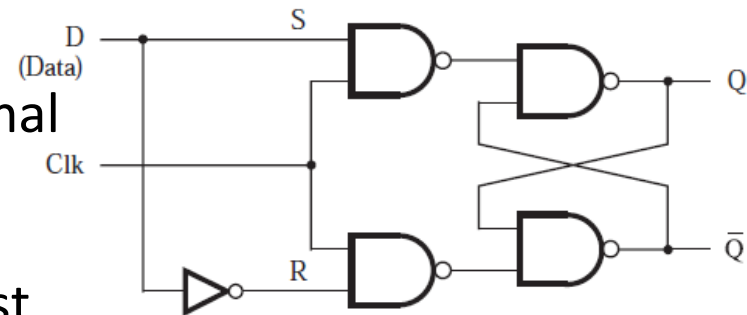




Propagation delays

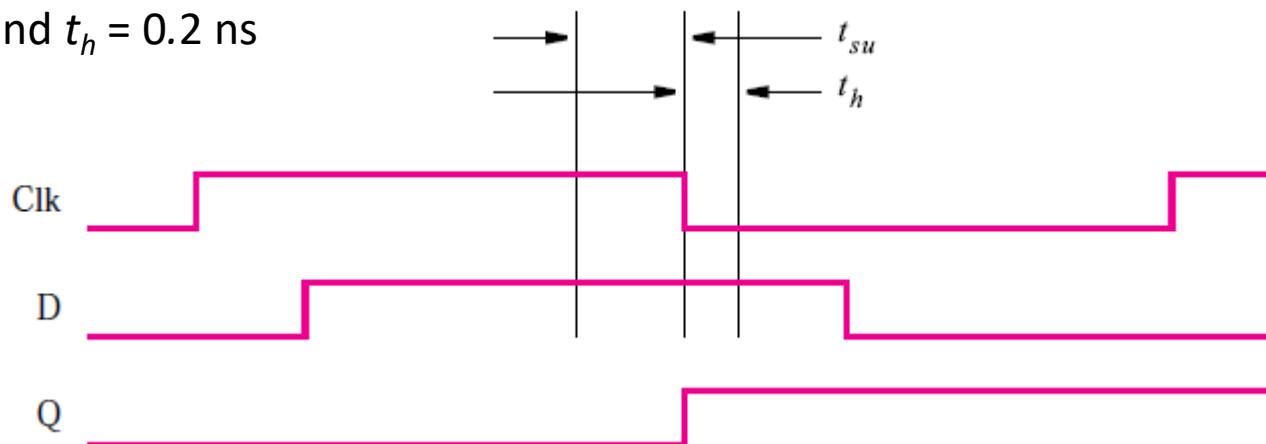
Effect of propagation delays must be considered in practice:

- Setup time: minimum time the D signal must be stable prior to clock transitioning from 1 to 0
- Hold time: minimum time D signal must be stable after negative edge of clock.



Typical values:

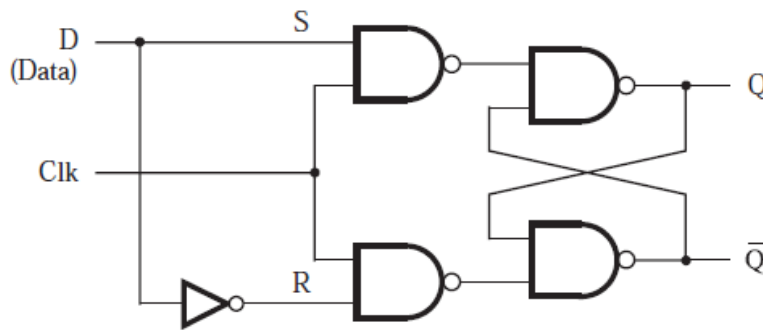
$t_{su} = 0.3$ ns and $t_h = 0.2$ ns





Level-Triggered vs. Edge-Triggered

Output of the D latch changes as the value of input changes while $\text{Clk} = 1$.



Clk	D	$Q(t + 1)$
0	x	$Q(t)$
1	0	0
1	1	1

- In principle, storage elements are used to hold output values of a circuit, which are inputs to another circuit.
 - Output value should not be stored in memory element until the “edge” of the clock
 - Called edge-triggered



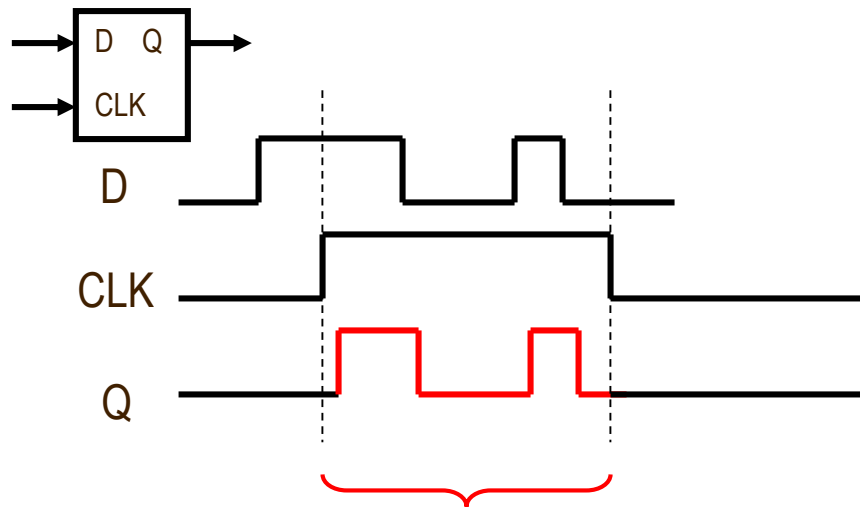
Latches vs. Flip Flop

Latches are level-sensitive: whenever clock is high, latch is transparent.

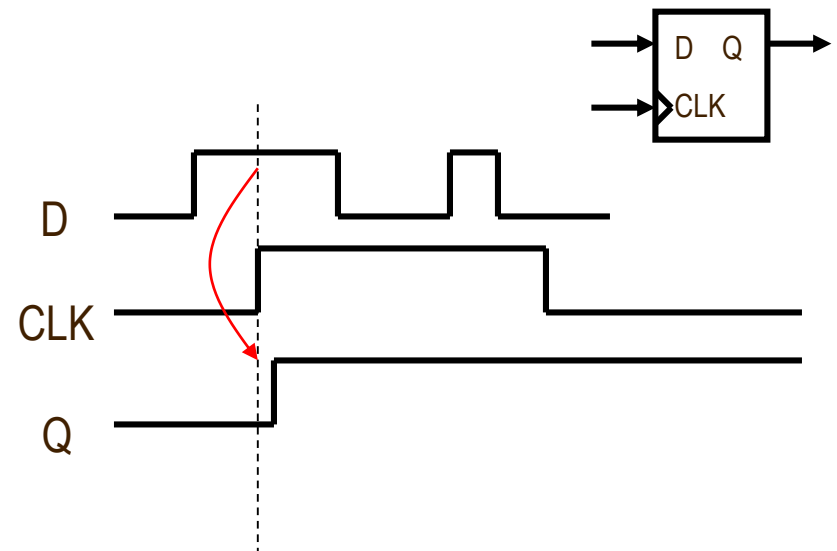
Flip-flops are edge-sensitive: data passes through (i.e. data is sampled) only on a rising (or falling) edge of the clock

Latches cheaper to implement than flip-flops

Flip-flops are easier to design than latches



Latch transparent when clock is high

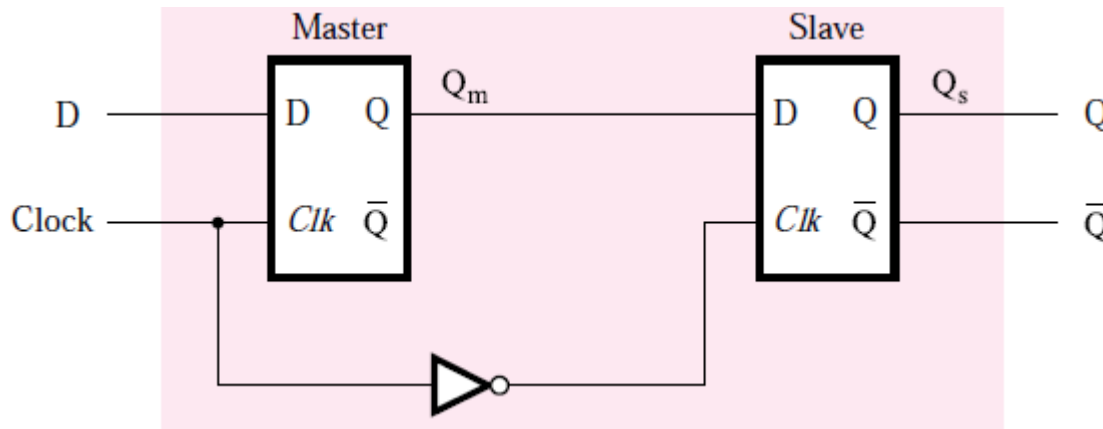


"Samples" D on rising edge of clock



4- Master-Slave D Flip-Flop (FFs)

- Flip-Flop: a storage element that changes its state at the **edge** of clock (output changes only once)



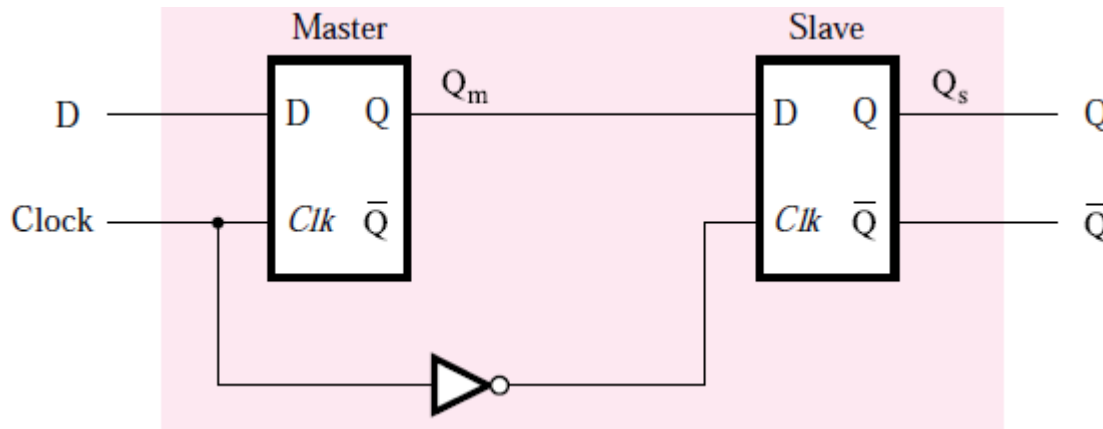
The **master**, changes its state while $C/k = 1$. The **slave** changes its state while $C/k = 0$.

When the clock signal changes to 0, the master stops following the changes in the D input. The slave responds to the value of the signal Q_m and changes state accordingly



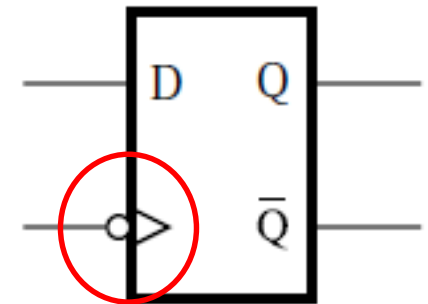
Master-Slave D Flip-Flop (FFs)

- Flip-Flop: a storage element that changes its state at the **edge** of clock (output changes only once)



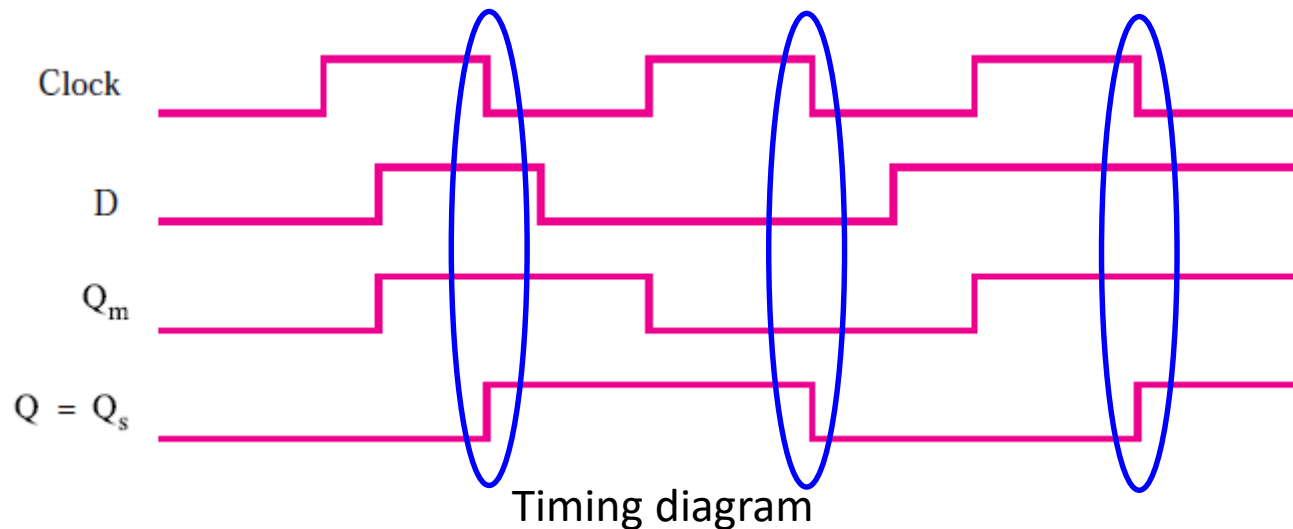
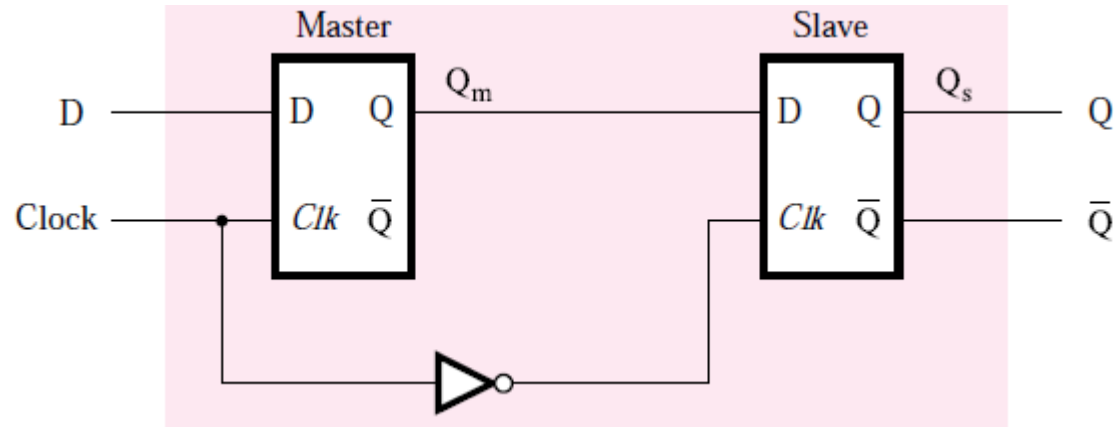
From an observer's point of view, the master-slave circuit changes its state at the **negative-going** edge of the clock (Clk goes from 1 $\rightarrow$ 0).

- Called negative-edged FF



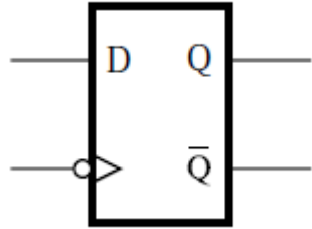


Master-Slave D Flip-Flop (FFs)

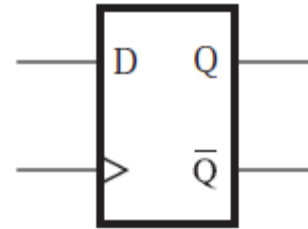




Negative-Edge vs. Positive-Edge



Negative-edge FF



Positive-edge FF

- Positive-edge FFs can be implemented using negative-edge FFs. How?
 - Invert the clock signal
- Other designs are possible with lower cost (fewer gates)



D FFs with Clear & Preset

Flip-flops often used for implementation of circuits that have many possible states.

Response of the circuit depends not only on the present values of the circuit's inputs but also on the particular state that the circuit is in at that time.

Example: A counter is a circuit that has a number of flip-flops, whose outputs are interpreted as a number.

The counter circuit has to be able to increment or decrement the number.

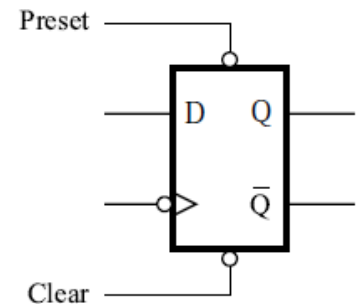
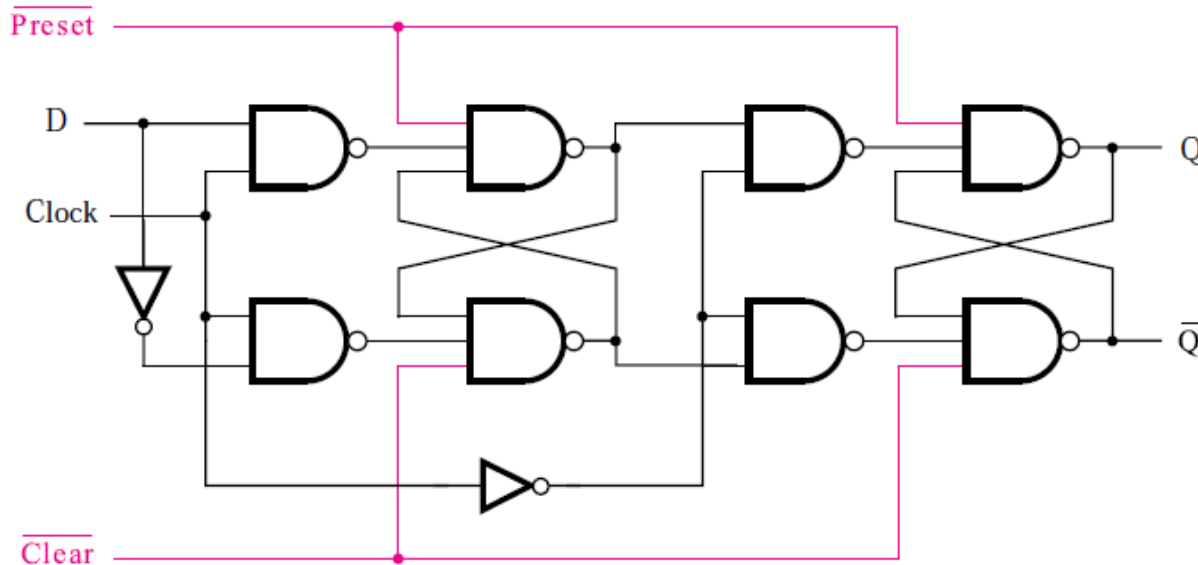
It is also important to be able to force the counter into a known initial state, i.e. clearing or setting the counter.



D FFs with Clear & Preset

Clear ($Q = 0$) and Preset ($Q = 1$) are needed:

- Clear/set storage element regardless of inputs
- Clear and preset should **not** be both 0 at the same time!!



- Clear = 0 $\rightarrow$ $Q = 0$
- Preset = 0 $\rightarrow$ $Q = 1$



Asynchronous/Synchronous Clear

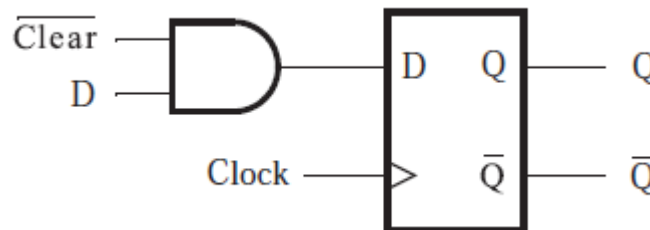
The effect of a low signal on either the *Clear* or *Preset* input is **immediate**.

For example, if *Clear* = 0 then the flip-flop goes into the state $Q=0$ immediately, regardless of the value of the clock signal.

We say flip-flop has an **asynchronous clear**.

On the other hand, *D* is **synchronous** (output changes at edge of the clock)

- Often preferable to clear the flip-flops on the active edge of the clock. I.e. FF has **synchronous clear**





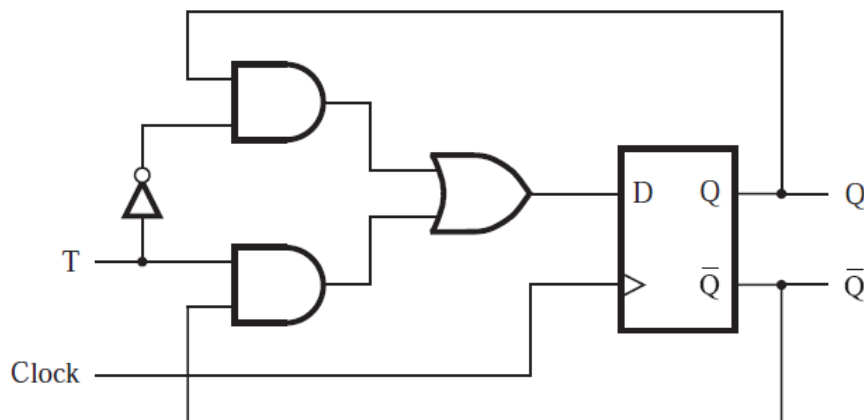
5- T Flip-Flop

T Flip-Flop: maintains current state or complements it, based on “Toggle” input T .

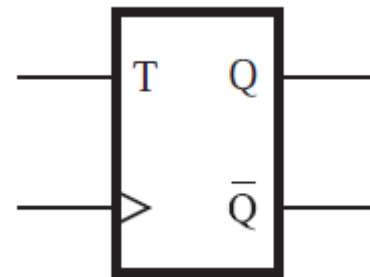
How can we turn the D flip-flop into a T one using simple logic?

Hint: Write down next state $Q(t+1)$ as a function of current state $Q(t)$ and input T .

● Answer: Set $D = T \cdot \bar{Q} + T \cdot Q$



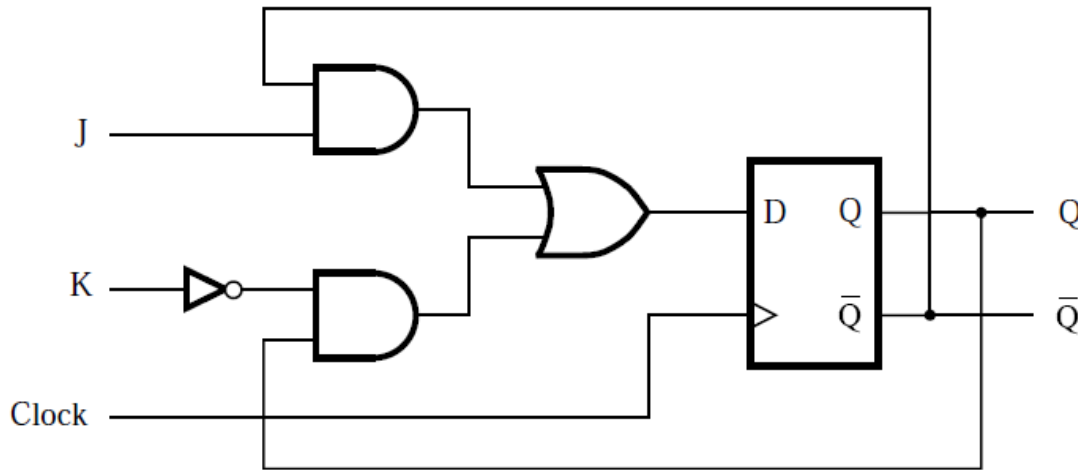
T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$



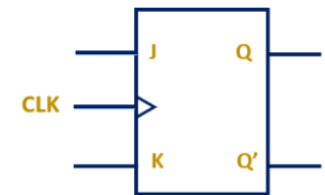


6- J-K Flip-Flop

How does the following circuit behaves? Fill in the characteristic table:



J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\bar{Q}(t)$



● Answer: $D = J.\bar{Q} + \bar{K}.Q$

- Combines the behaviors of SR and T flip-flops. Behaves as the SR flip-flop, where $J = S$ and $K = R$, for all input values except $J=K=1$, where it toggles its state like the T flip-flop.



Summary of Terminology

Basic Latch: SR

Gated Latch

Gated SR latch (Positive level and negative level)

Gated D latch (Positive level and negative level)

Flip-Flop (it is called Master-slave and it is edge triggered)

D FF

T FF

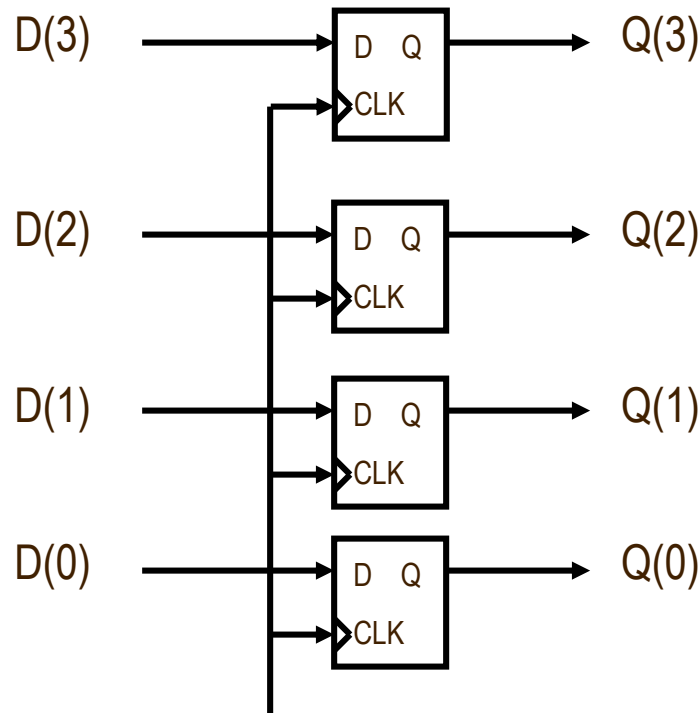
JK FF



Registers

A flip-flop stores one bit of information. When a set of n flip-flops is used to store n bits of information, we refer to these flip-flops as a *register*. A **common clock** is used for a register.

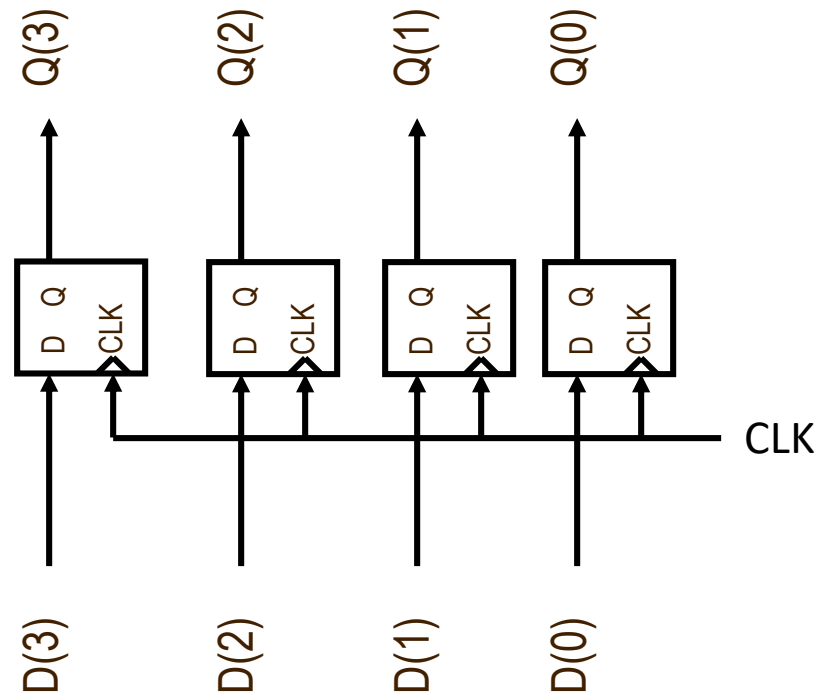
A register may have additional control signals such as Enable, Clear.





Parallel Load Registers

Design a parallel load register of 4 bits using D FFs.

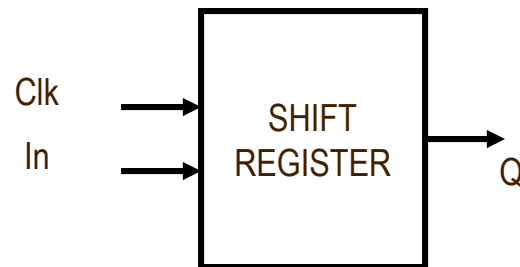
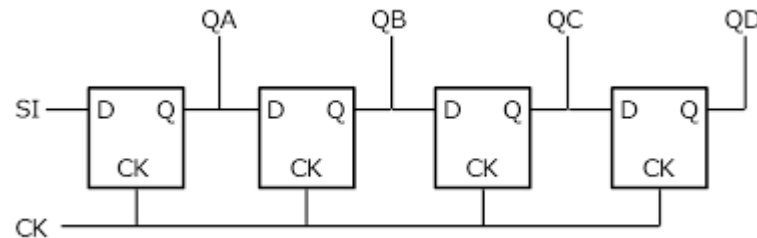




Shift Registers

Shift register (bits are shifted one bit at a time from left to right or from right to left).

Information is loaded **serially**

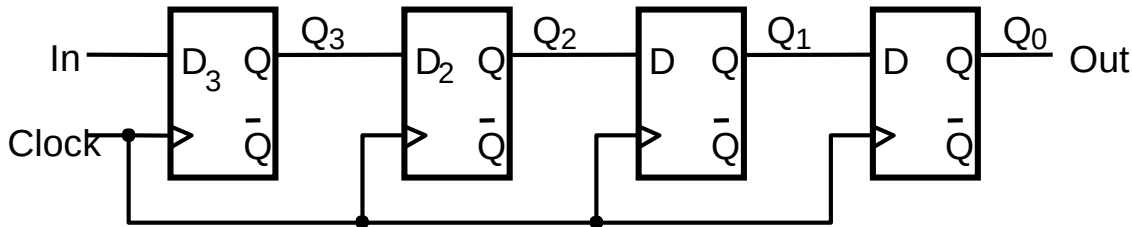




Right Shift Registers

Design 4 bit right shift register using D FFs.

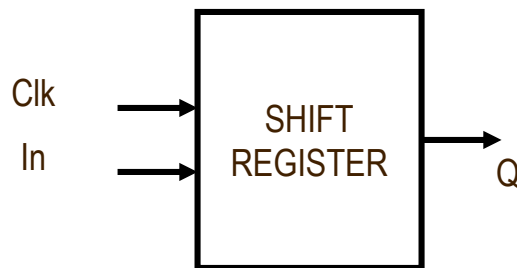
Information is loaded **serially**



(a) Circuit

	In	Q ₃	Q ₂	Q ₁	Q ₀ = Out
t_0	1	0	0	0	0
t_1	0	1	0	0	0
t_2	1	0	1	0	0
t_3	0	1	0	1	0
t_4	1	0	1	0	1

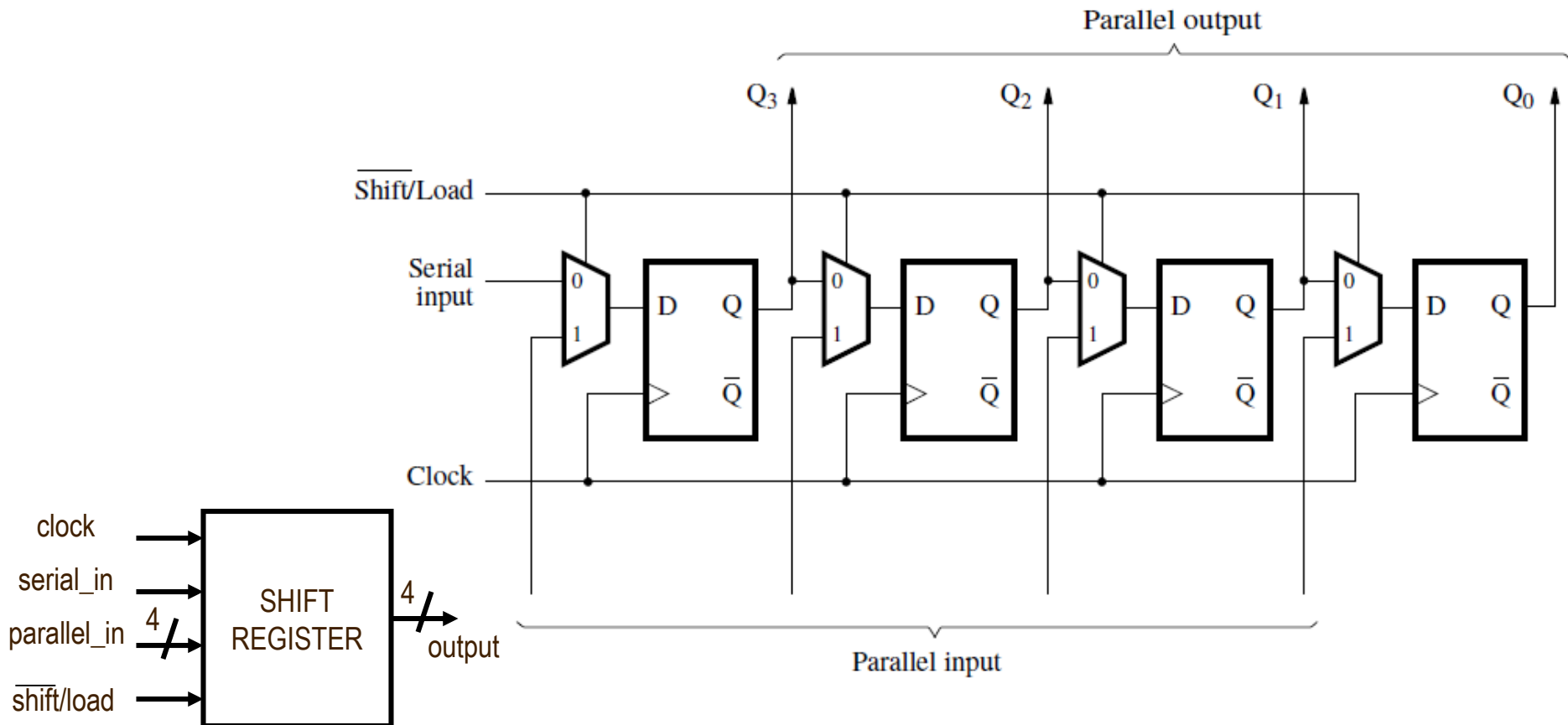
(b) A sample sequence





Parallel Load and Shift Register

Serial to parallel data transfer or parallel to serial data transfer. It is used in data communications. The shift/load control signal is used to select the mode of operation.





Counters

Counter circuits (increment or decrement a count by 1) are used in digital systems for many purposes.

- Count the number of occurrences of certain events,
- Generate timing intervals for control of various tasks in a system,
- Keep track of time elapsed between specific events, and so on.

Can be implemented with adder/subtractor circuits, how?

Better use simpler circuits that have a significantly lower cost, e.g. (using D/T FFs)

Asynchronous counters: **no common clock**

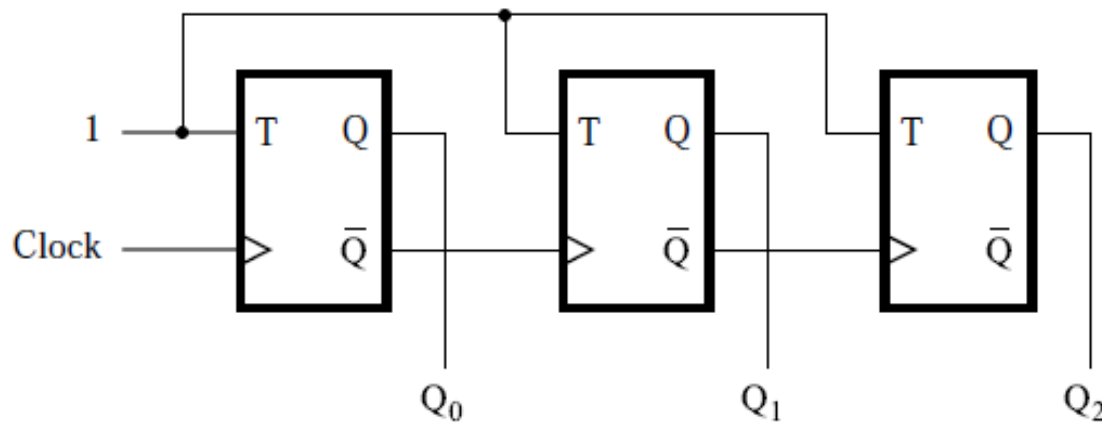
Synchronous counters have **a common clock**



Asynchronous Counters

Example: 3-bit up counter using T FFs. How can we build it using 3 FFs?

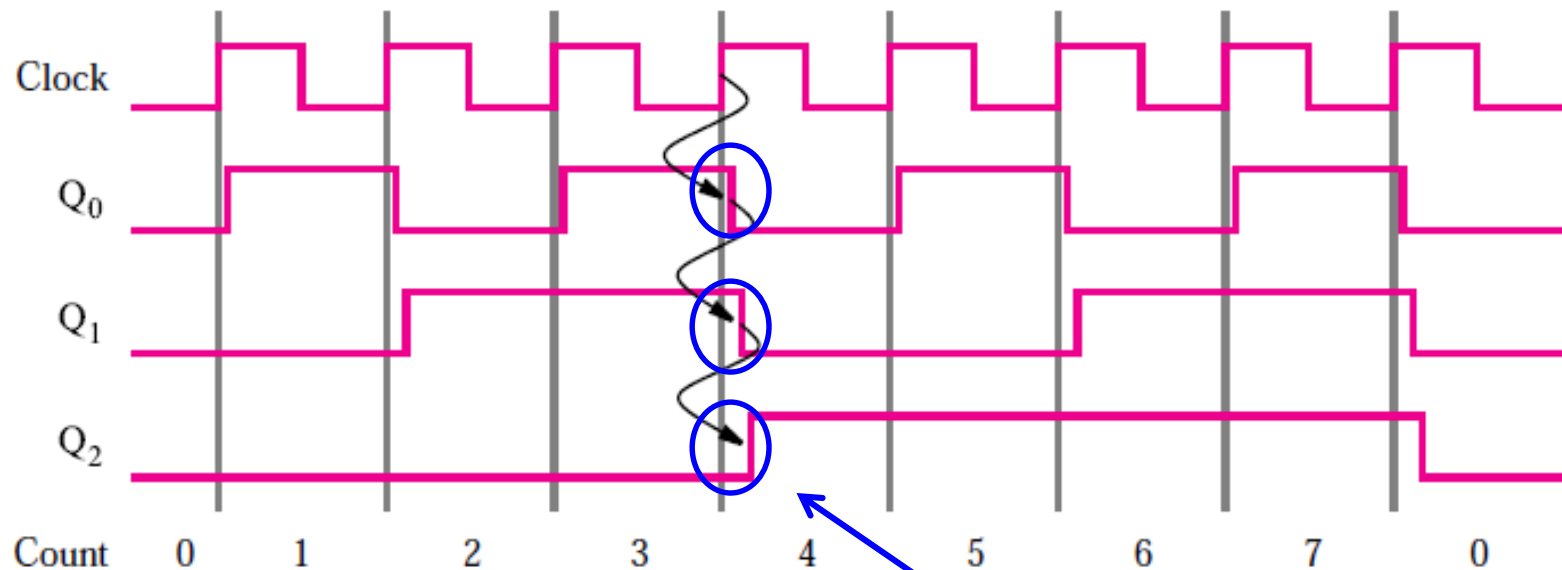
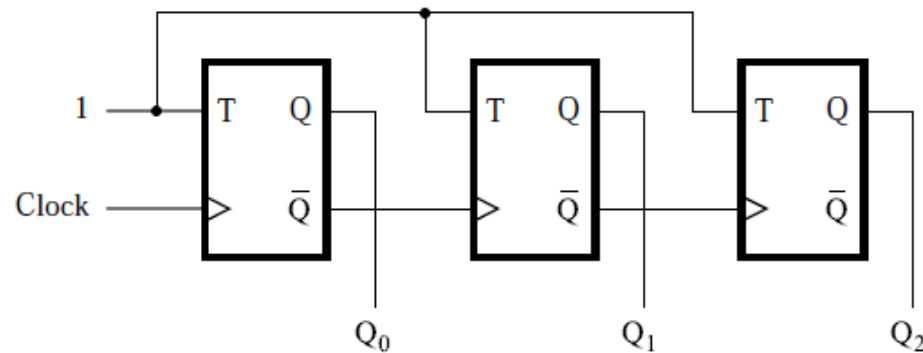
Hint: Think how the bits of a number change as we count from zero to seven.



- The last two flip-flops have their clock inputs driven by the Q output of the preceding flip-flop.
 - Therefore, they toggle their state whenever the preceding flip-flop changes its state from $Q = 1$ to $Q = 0$



Up-Counter (0-to-7) Waveform

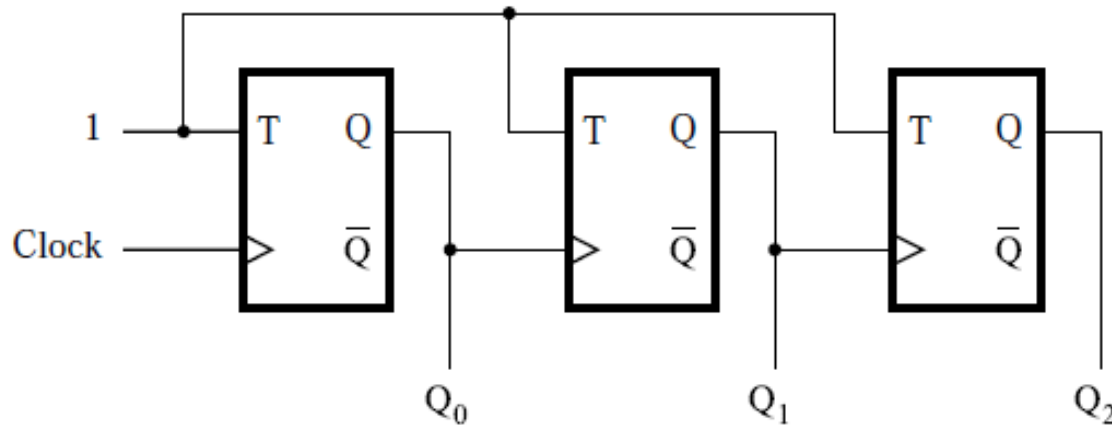


Delay- Ripple Counter!



Another counter

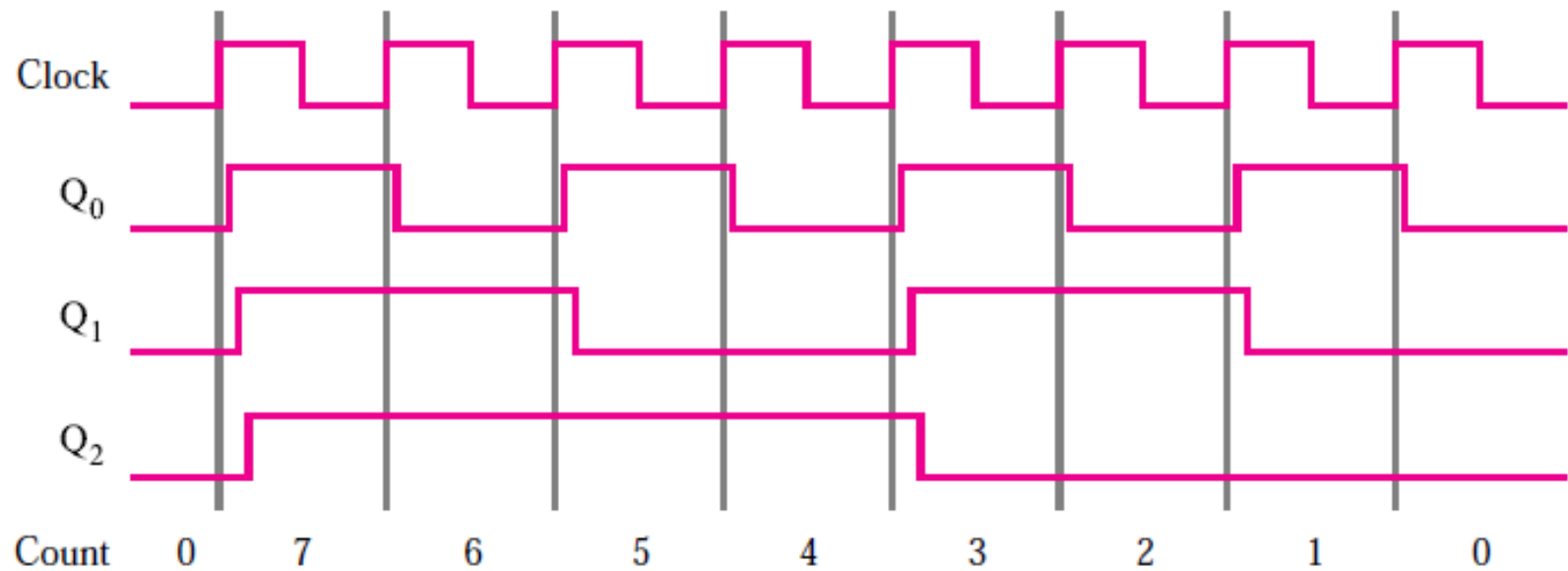
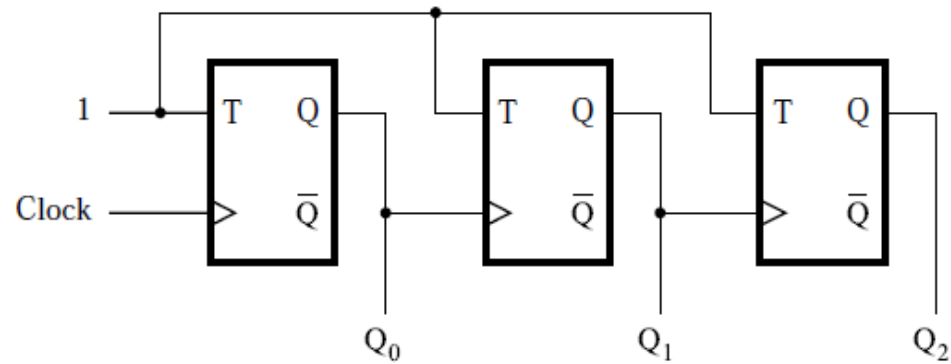
- What does the following circuit do?



- It's a **down-counter** (from 7 to 0).
 - The last two flip-flops have their clock inputs driven by the Q output of the preceding flip-flop.
 - Therefore, they toggle their state whenever the preceding flip-flop changes its state from $Q = 0$ to $Q = 1$



Down-Counter (7-to-0) Waveform

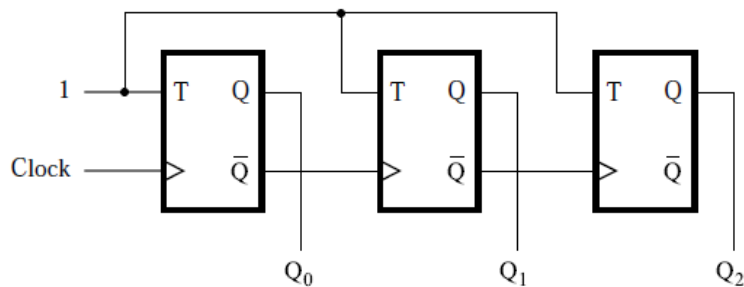




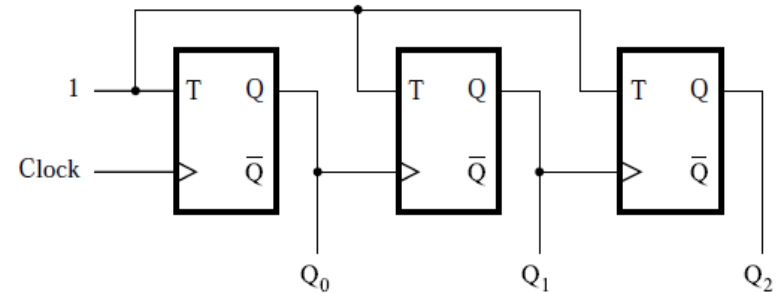
Up-Down counter

It is possible to combine the functionalities of the previous two circuits to form a counter that can count either up or down.

How?



Up counter



Down counter



Synchronous up-counter

The asynchronous counters are simple but not fast.

Delays may become too long!

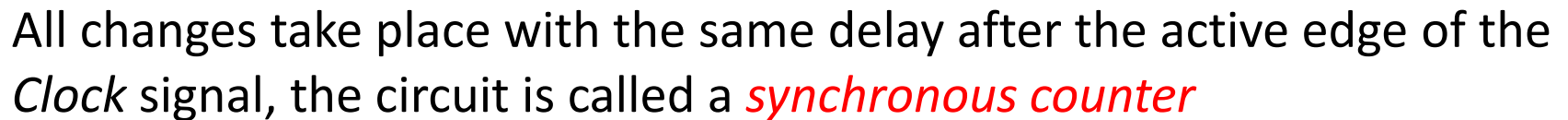
We can build a faster counter by clocking **all** flip-flops at the same time.

Clock cycle	Q ₂	Q ₁	Q ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Q₁ changes

Q₂ changes

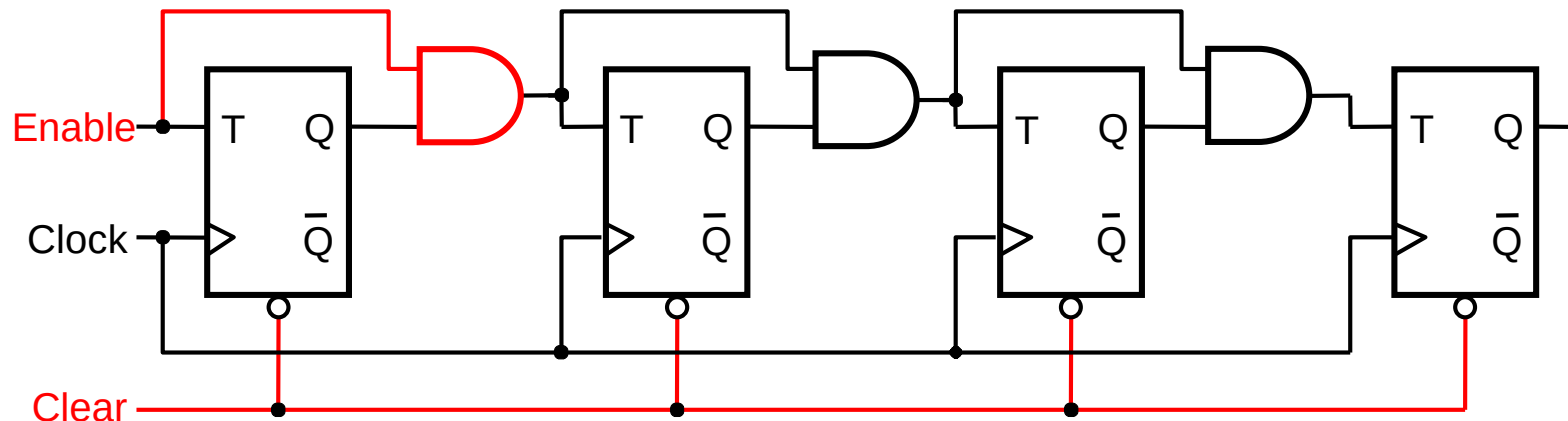
Fact: A FF changes its state only when ALL previous FFs are in state Q=1





Counter w/ Enable & Clear

Add more functionality to synchronous counter:
The Enable signal “freezes” counting
Clear signal resets all FFs to 0





Synchronous counter with D FFs

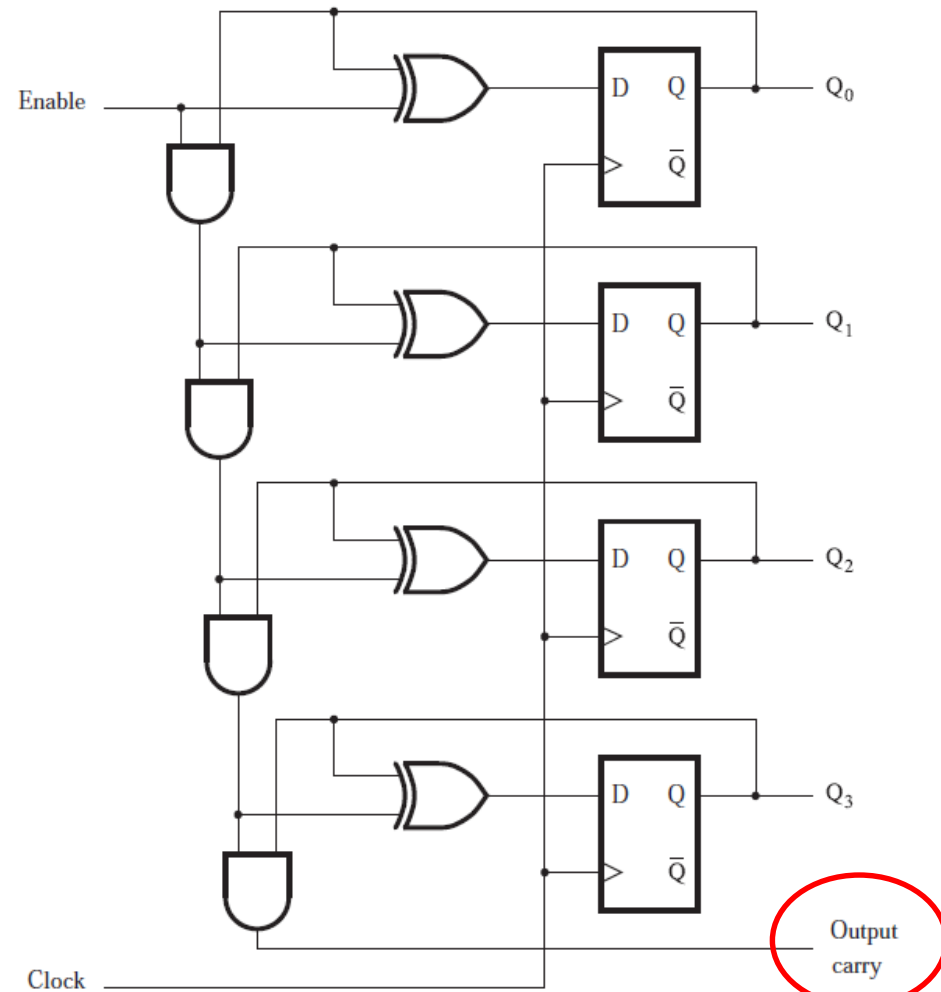
It is also possible to build counters using other types of FFs.

JK flip-flops same as the T flip-flops by setting $J = K$.

We will now consider using D flip-flops for this purpose.

Recall, a FF must change its state if ALL the previous FFs have Q=1.
Use XOR!

$$D_i = Q_i \oplus Q_{i-1}Q_{i-2} \cdots Q_1Q_0$$





Counters w/ Parallel Load

Often it is desirable to start with a different count.

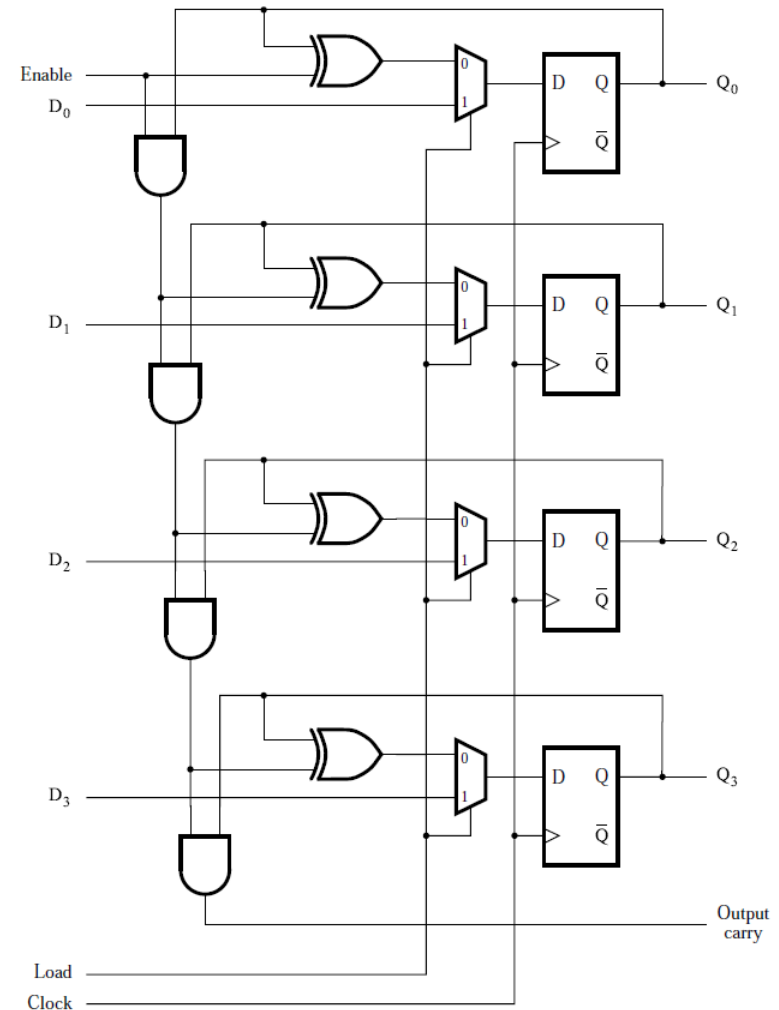
Counter must have some inputs through which the initial count can be loaded.

A 2-1 multiplexer can be used before each D input.

A control signal, Load, is used to choose the mode of operation.

Count operation (Load = 0)

Load operation (Load = 1).



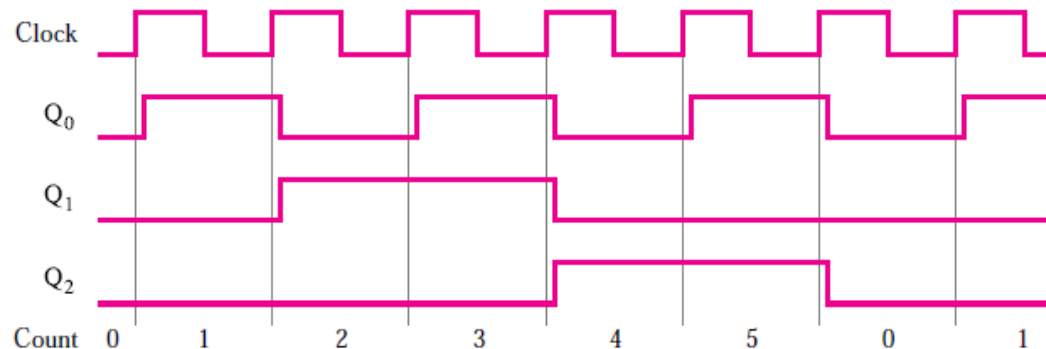
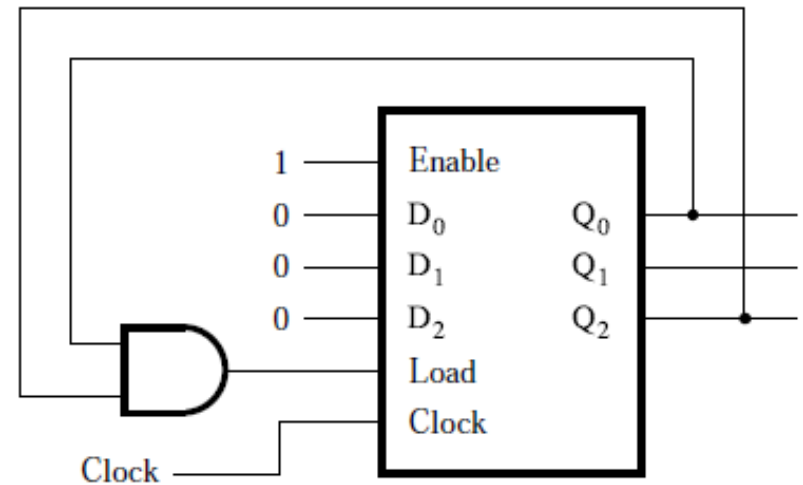


Reset Synchronization

Interested in resetting the counter to 0 during the normal counting process.

Example: Construct a Modulo-6 counter. Reset should occur after counter reaches 101.

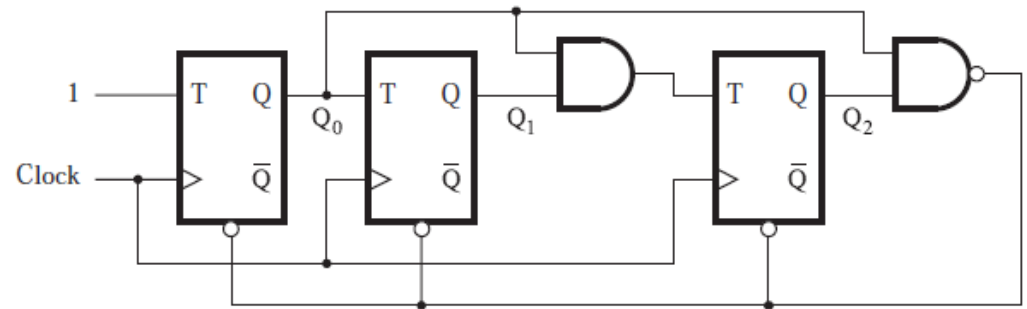
Idea: Load 000 when this case occurs



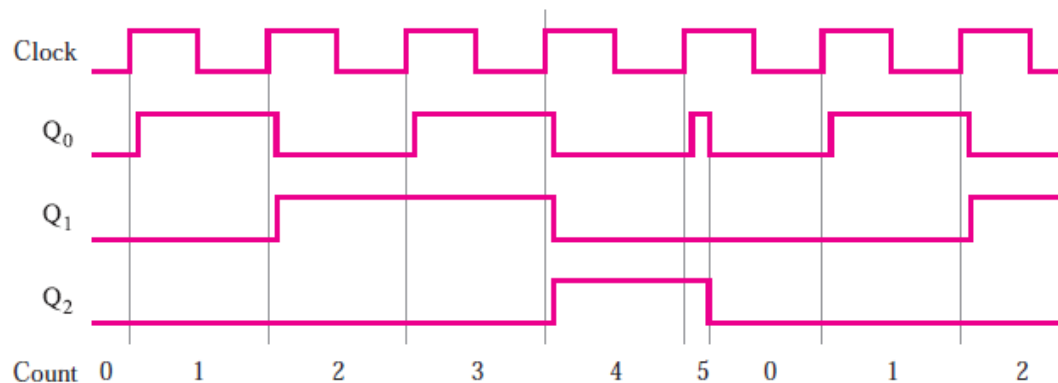


Asynchronous Reset Problem

Previous counter was reset on the active edge of the clock (**synchronous reset**).



- Consider now the possibility of using the clear feature of individual flip-flops, rather than the parallel-load approach (**asynchronous reset**).



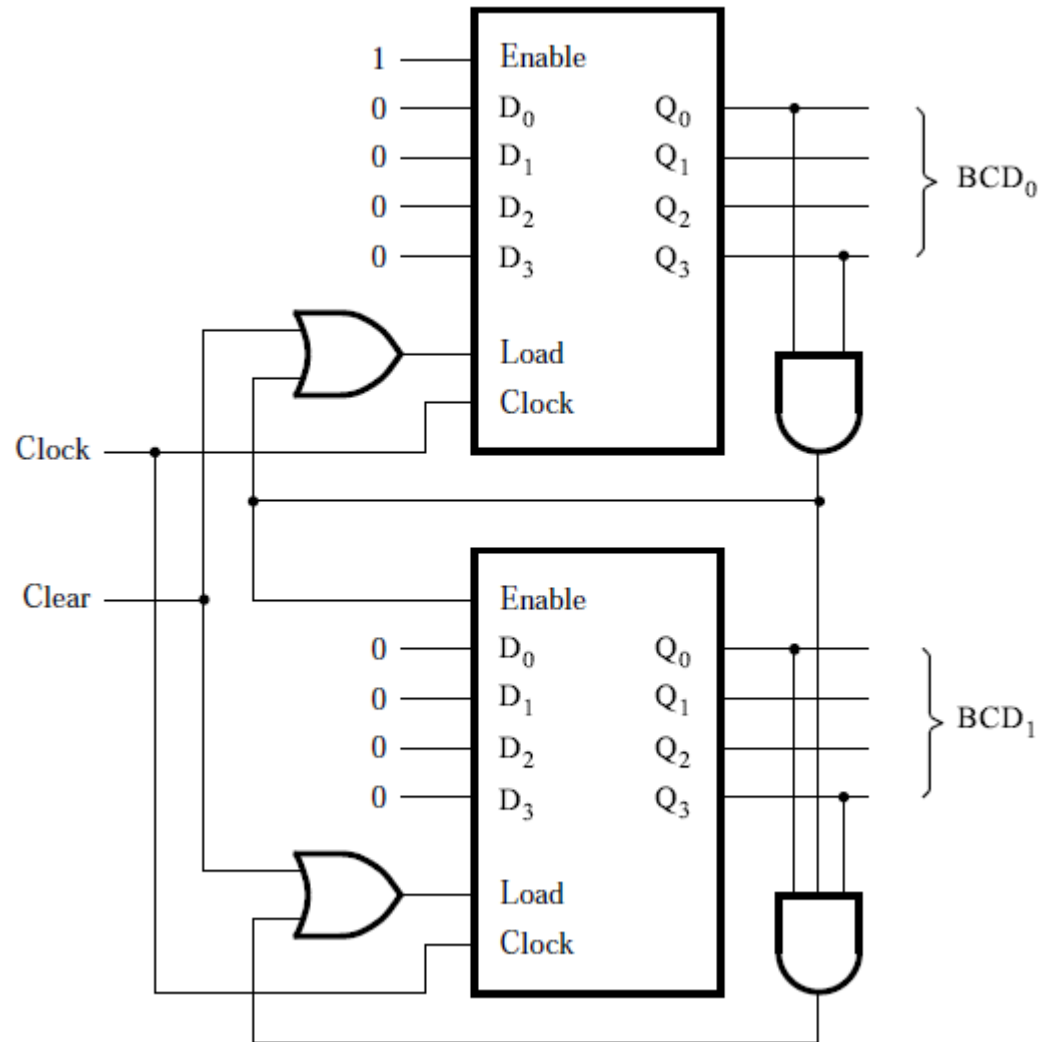


BCD counter

2 digit Modulo-10 counter

BCD numbers are 0-to-9

When the count in BCD_0 reaches 9 (1001), the second stage is enabled



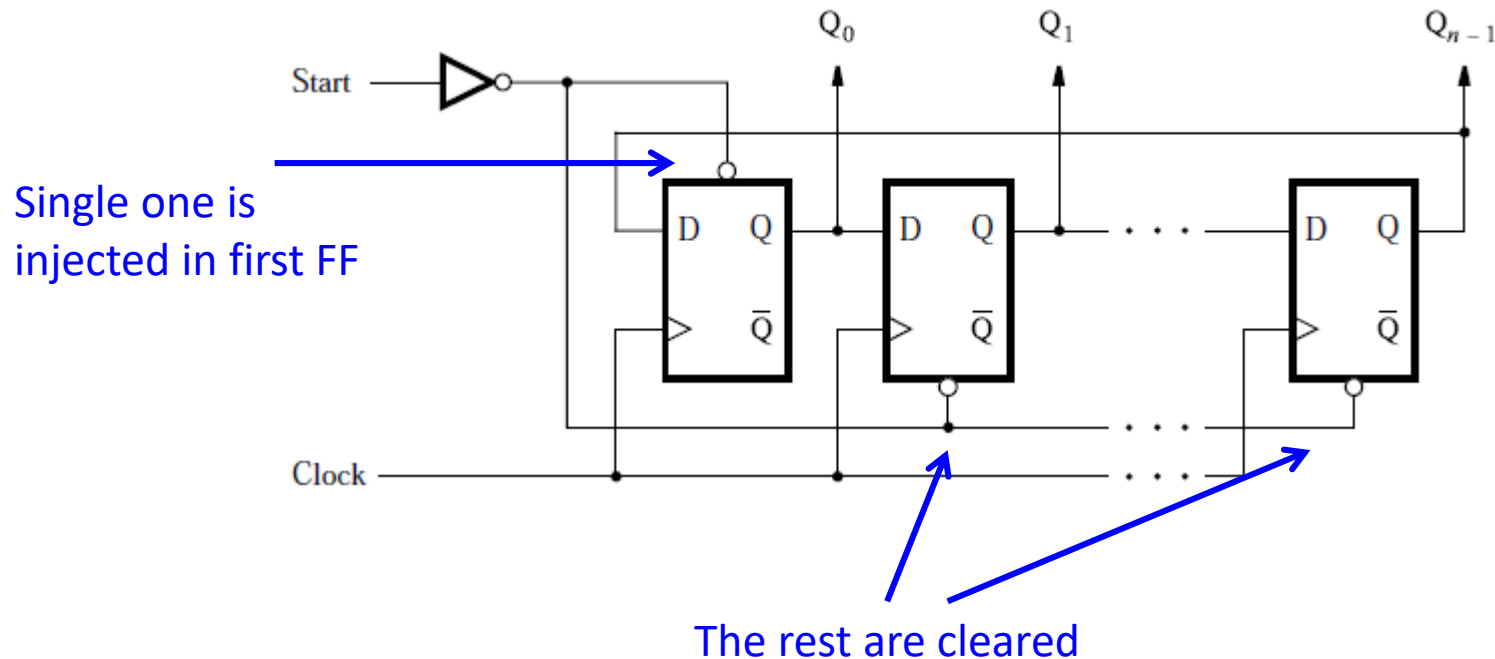


Ring counter

Outputs the *one-hot* code sequence

1000, 0100, 0010, 0001, 1000,...etc

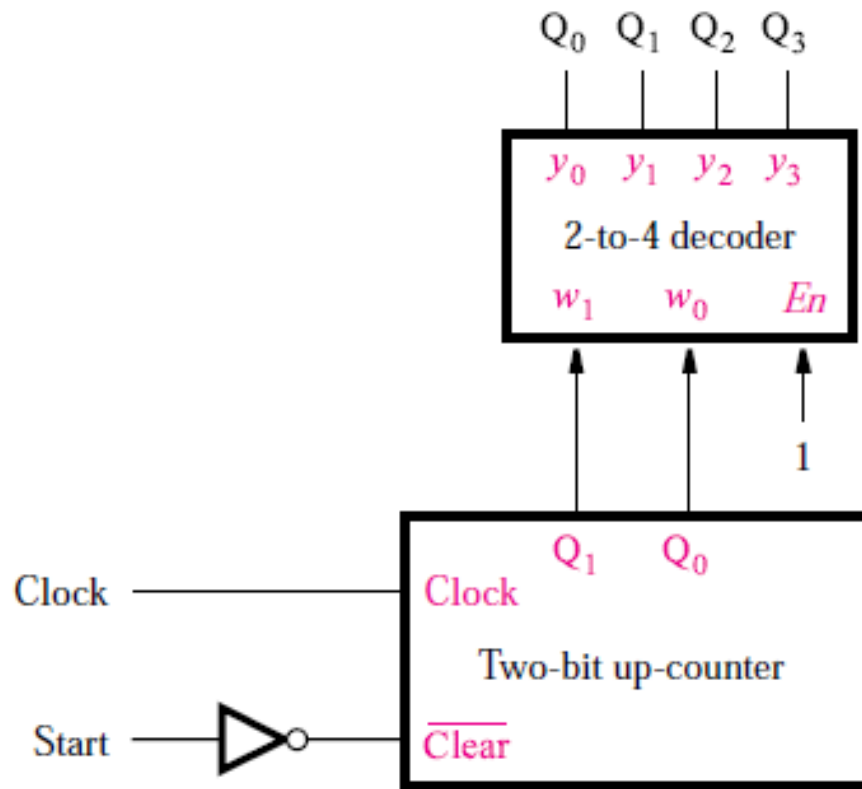
Can be implemented with a shift register in which a single 1 is injected.





Ring counter

How can the same counter be implemented with a 2-bit up counter and a 2-to-4 decoder?





Johnson Counter

- Counts this way $Q_3Q_2Q_1Q_0$:
 - 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000, 0000, 0001, ...etc
- Only one bit changes at a time
- Connects Q' as input to the first stage

